Codigo modificado con mutex.

(como el documento no especificaba que usaramos justamente el mismo que hay ahí, yo hice mi propia versión)

```
#include <iostream>
#include <thread>
#include <mutex>

int contador = 0;
std::mutex mutexContador;

void procesoA()
{
    for (int i = 0; i < 100000; i++)
    {
        mutexContador.lock();
        contador++;
        mutexContador.unlock();
    }
}

void procesoB()
{
    for (int i = 0; i < 100000; i++)
    {
        mutexContador.lock();
        contador++;
        mutexContador.unlock();
    }
```

```
}

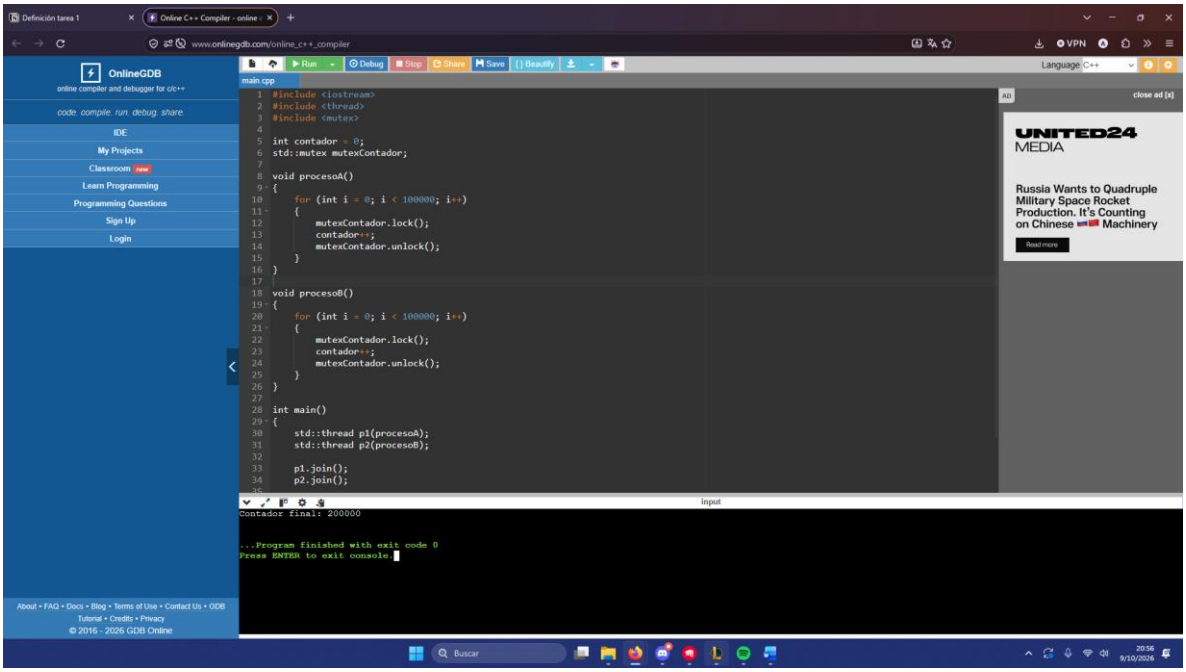
int main()
{
    std::thread p1(procesoA);
    std::thread p2(procesoB);

    p1.join();
    p2.join();

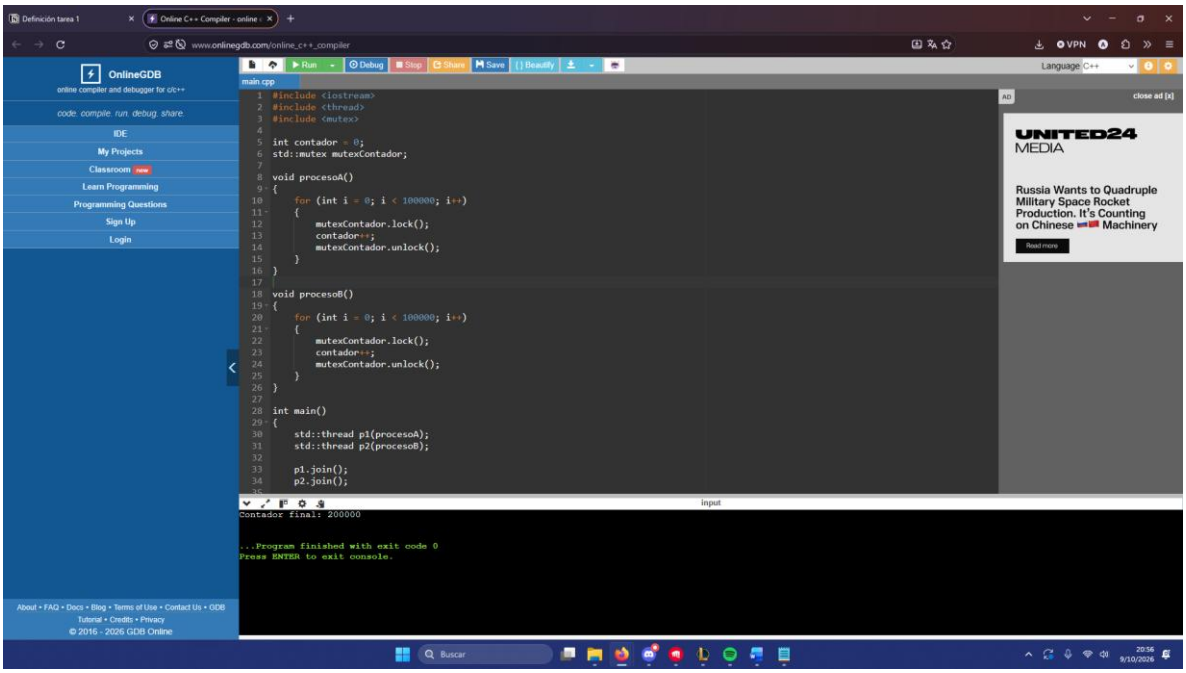
    std::cout << "Contador final: " << contador << std::endl;

    return 0;
}
```

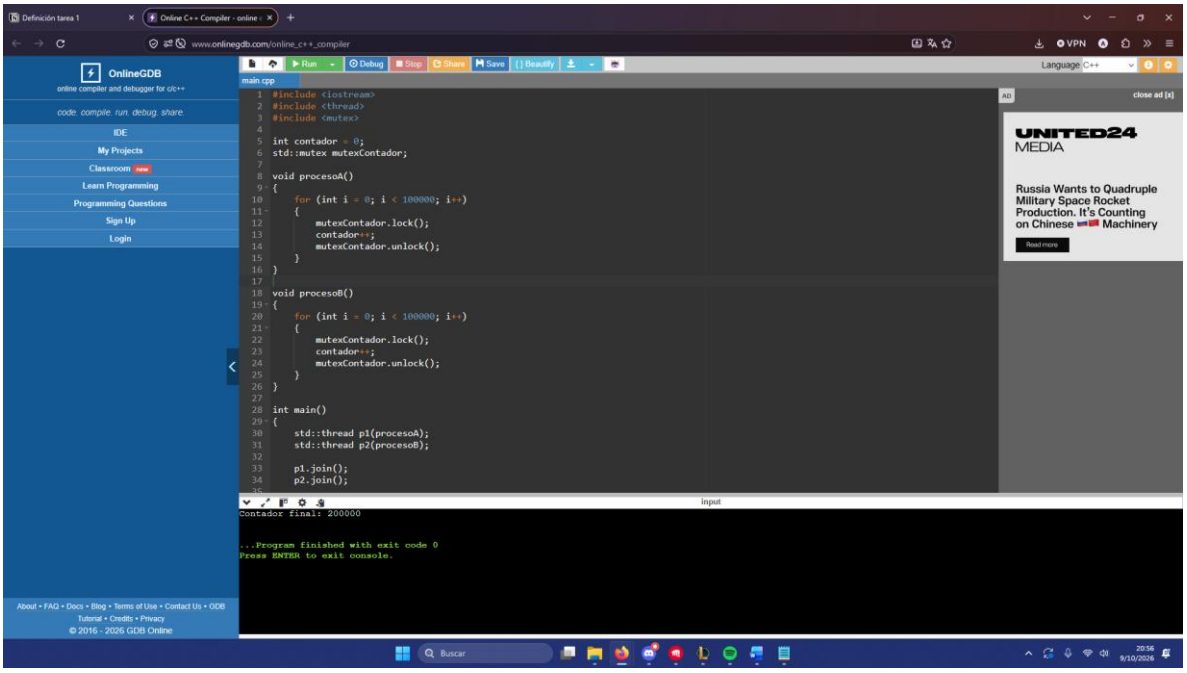
Primera ejecución



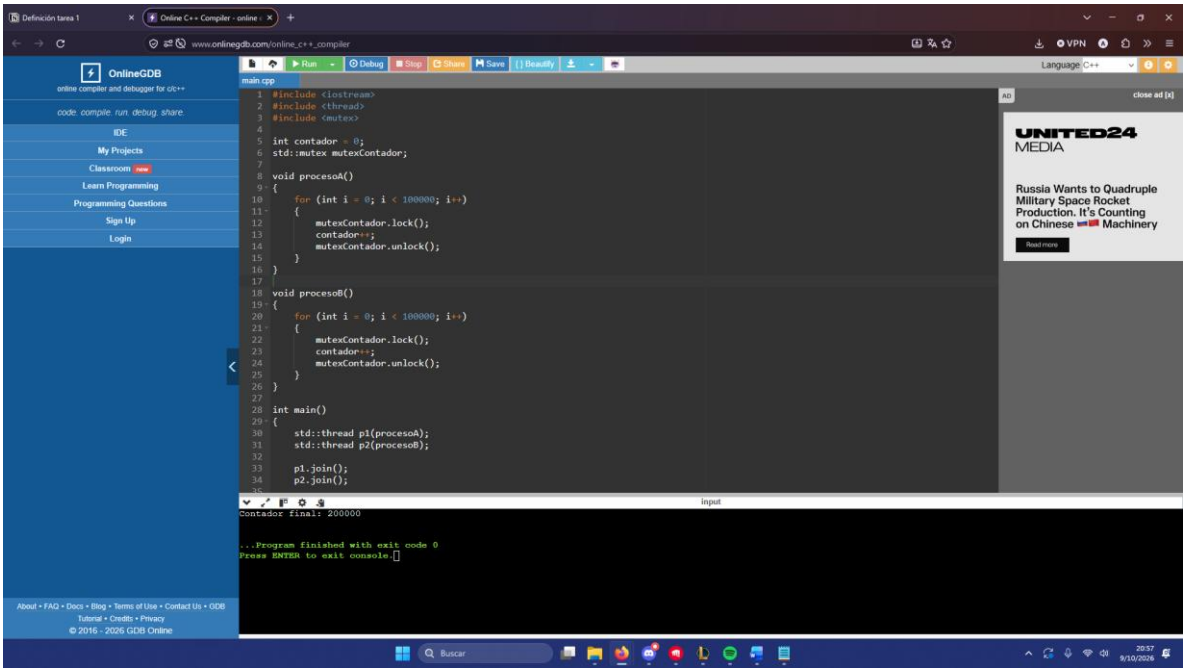
Segunda ejecución



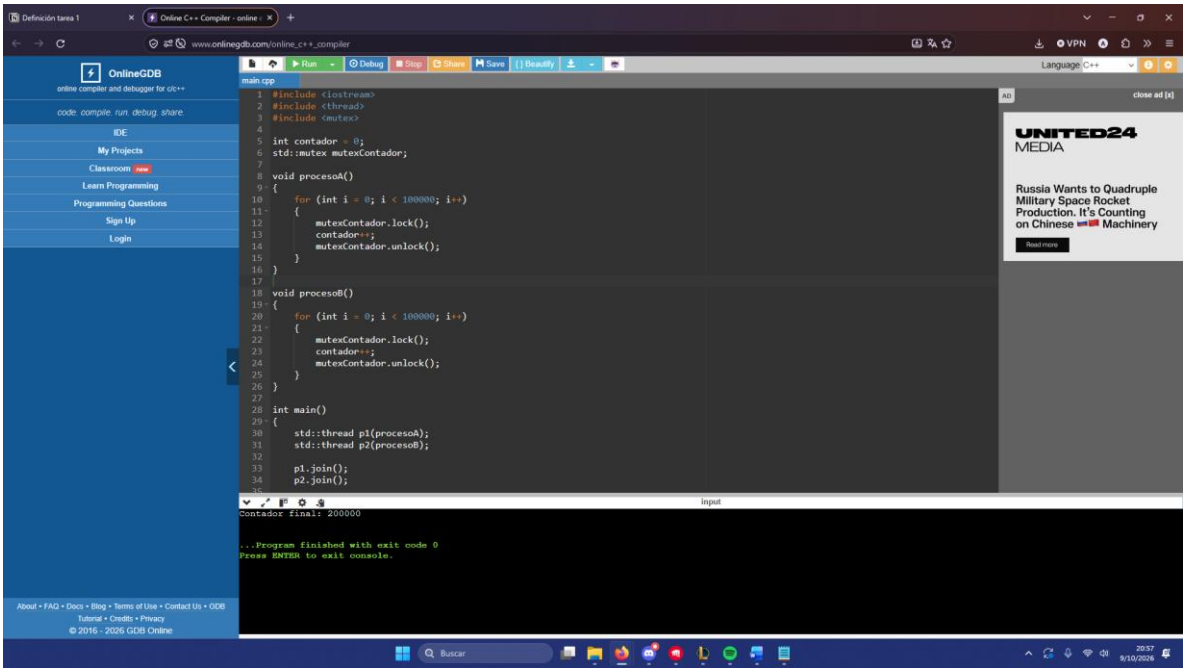
Tercera ejecución



Cuarta ejecucion



Quinta Ejecucion



Ejecución	Valor esperado	Valor obtenido
1	200000	200000
2	200000	200000
3	200000	200000
4	200000	200000
5	200000	200000

Identifique exactamente qué instrucciones forman parte de la sección crítica.

La sección crítica es la instrucción contador++, ya que modifica una variable compartida entre los dos hilos. Esta operación implica leer el valor actual, incrementarlo y escribir el nuevo valor. Por eso, debe protegerse mediante mutexContador.lock() y mutexContador.unlock() para evitar que ambos hilos la ejecuten simultáneamente

Explique cómo el mutex evita que los dos procesos modifiquen simultáneamente el contador.

El mutex permite que solamente un hilo acceda a la sección crítica a la vez. Cuando un hilo ejecuta lock(), adquiere el bloqueo y el otro debe esperar hasta que se ejecute unlock(). De esta manera, los incrementos se realizan sin interferencias entre los hilos, evitando la condición de carrera y garantizando que el contador alcance el valor esperado de 200000

Comparación entre el programa original y el programa con mutex

En el programa original, los dos hilos incrementaban la variable contador sin ningún mecanismo de sincronización. Esto provocaba una condición de carrera, ya que ambos podían leer y modificar el mismo valor simultáneamente, ocasionando la pérdida de algunos incrementos. Durante las cinco ejecuciones, los resultados variaron entre 105803 y 145720, sin alcanzar el valor esperado de 200000.

En cambio, el programa modificado utiliza std::mutex para proteger la sección crítica mediante las instrucciones lock() y unlock(). Esto permite que solamente un hilo modifique el contador a la vez, evitando las interferencias entre ambos. Como resultado, las cinco ejecuciones del programa modificado alcanzaron correctamente el valor de 200000.

En conclusión, la principal diferencia entre ambos programas es la sincronización de los hilos. El programa original presenta una condición de carrera que genera resultados incorrectos, mientras que el uso de mutex garantiza que los incrementos se realicen correctamente y que el contador alcance el valor esperado.